

Week 41: Backpropagation in General, and a Neural Network of Our Own

The four equations, the cost functions for regression and classification – and a code that reads 70 000 handwritten digits

With pauses for questions

Morten Hjorth-Jensen

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway

Plan for week 41

Monday lecture (2×45 min):

- ▶ **First 15 min, the reminder from last week:** the three smallest networks – no hidden layer, one hidden neuron, two inputs and two hidden nodes – the error $\delta = \partial C / \partial z$, the rule *gradient = error times the signal that entered the weight*, and the training program of the notes that we generalise today (Sections 8.8.1–8.8.2)
- ▶ **Backpropagation in general:** dense layers, the fan-out sum, the four equations and the algorithm, why it costs two forward passes (Sections 8.8.3–8.8.5), and the two things a code must get right: vanishing gradients and initialisation (8.9–8.10)
- ▶ **Cost functions and output layers:** the squared error with ℓ_2 and ℓ_1 penalties, the cross entropy for binary classification, the softmax and the multiclass cross entropy for MNIST – and why all three give the same output error (8.7, 8.11–8.12, Chapter 5)
- ▶ **Our own code:** the template of the notes with a loop over layers, checked against finite differences and `jax.grad`, on the Runge function, on a binary problem and on the full MNIST set (8.13–8.14)

Tuesday (flipped classroom): your questions, then *one* case: the feed-forward part of a network, step by step – the structure of this week's exercises (**deadline Friday October 9**).

Reading: Chapter 8, Sections 8.8–8.14; Sections 5.3 and 5.6 for the cross entropy and the softmax. Goodfellow et al. Chapter 6; Raschka et al. Chapter 11; Nielsen, *Neural Networks and Deep Learning*, Chapter 2.

Where we were: three small networks, one rule (Sections 8.8.1–8.8.2)

No hidden layer, Eqs. (8.35)–(8.36): $z^1 = w^1 x + b^1$, $a^1 = \sigma(z^1)$, $C = \frac{1}{2}(a^1 - y)^2$,

$$\delta^1 = \frac{\partial C}{\partial z^1} = (a^1 - y) \sigma'(z^1), \quad \frac{\partial C}{\partial w^1} = \delta^1 x, \quad \frac{\partial C}{\partial b^1} = \delta^1.$$

One hidden neuron, Eq. (8.43): the output layer is the same formula with a^1 in place of x , and the hidden error is $\delta^1 = \delta^2 w^2 \sigma'(z^1)$ – the reverse sweep of Section 4.14 that did not stop at the output.

Two inputs, two hidden nodes, Eqs. (8.52)–(8.55): the same with indices,

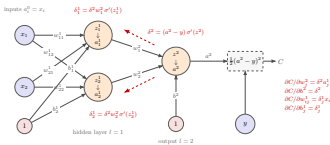
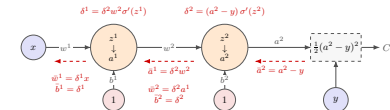
$$\delta_j^1 = \delta^2 w_j^2 \sigma'(z_j^1), \quad \frac{\partial C}{\partial w_{ij}^1} = \delta_j^1 a_i^{1-1}, \quad \frac{\partial C}{\partial b_j^1} = \delta_j^1,$$

or $\delta^1 = (W^2 \delta^2) \circ \sigma'(z^1)$ and $\partial C / \partial W^1 = x(\delta^1)^T$.

The rule

The error of a node is the derivative of the cost with respect to its pre-activation. The gradient of a weight is the error at the node it arrives at times the signal at the node it leaves; the gradient of a bias is the error itself. Everything needed on the way back was stored on the way forward.

one hidden layer, Eq. (8.40)



Book Figures 8.10 (bottom) and 8.12: forward black, reverse sweep red.

What was missing: a node that feeds *several* nodes (Pause and think 5 of week 40)

In the 2-2-1 network every hidden node has one successor

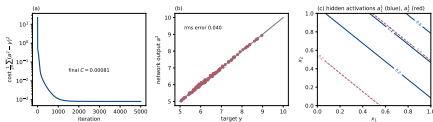
z_j^1 reaches C along a single path, through z^2 via w_j^2 , so its adjoint arrives in one term: $\delta_j^1 = \delta^2 w_j^2 \sigma'(z_j^1)$. No sum was needed, and that is the only reason the three examples fit on one slide.

Give it a second output node

Now z_j^1 reaches C along two paths, through z_1^2 via w_{j1}^2 and through z_2^2 via w_{j2}^2 , and the chain rule *adds* them:

$$\delta_j^1 = (\delta_1^2 w_{j1}^2 + \delta_2^2 w_{j2}^2) \sigma'(z_j^1) = \left(\sum_k \delta_k^2 w_{jk}^2 \right) \sigma'(z_j^1).$$

The fan-out rule of Section 4.14.4, as the two branches of Figure 5.5 were summed at v_2 . With M_{l+1} successors the sum has M_{l+1} terms – and that sum is the whole of today's Part 2.



The 2-2-1 network of week 40 on $y = x_1^2 + 3x_1x_2 + x_2^2 + 5$ (week40.ipynb): cost 22.8 $\rightarrow$ 0.00081, two sigmoid ridges combined by a linear output node. Nine parameters; today's MNIST network has 79 510.

The template we generalise today: the training program of the notes (week41.do.txt)

```
def forwardpropagation(x):
    z_1 = np.matmul(x, w_1) + b_1 # hidden pre-activation
    a_1 = sigmoid(z_1)
    z_2 = np.matmul(a_1, w_2) + b_2 # output pre-activation
    a_2 = z_2 # linear output
    return a_1, a_2

def backpropagation(x, y):
    a_1, a_2 = forwardpropagation(x)
    delta_2 = a_2 - y # f' = 1
    delta_1 = np.matmul(delta_2, w_2.T) * a_1 * (1 - a_1)
    output_weights_gradient = np.matmul(a_1.T, delta_2)
    output_bias_gradient = np.sum(delta_2, axis=0)
    hidden_weights_gradient = np.matmul(x.T, delta_1)
    hidden_bias_gradient = np.sum(delta_1, axis=0)
    return output_weights_gradient, output_bias_gradient, hidden_weights_gradient, hidden_bias_gradient

gamma = 0.1 # x = 4, y = 2x + 1, seed 0
for i in range(50):
    derivW2, derivB2, derivW1, derivB1 = backpropagation(x, y)
    w_2 -= gamma * derivW2; b_2 -= gamma * derivB2
    w_1 -= gamma * derivW1; b_1 -= gamma * derivB1
```

Last week's numbers: cost $36.90 \rightarrow 0.427 \rightarrow 0.0049 \rightarrow 10^{-7}$ after 0/10/20/49 iterations, $w^1 = 1.796$, $w^2 = 4.695$.

Four decisions already taken in these lines

Samples are rows: x has shape $(n, \text{features})$, the forward pass reads $x @ w_1 + b_1$, Eq. (8.21), so W^I has shape (inputs, outputs). The forward pass stores a_1 for the backward pass. Pull back, then modulate: $\text{delta}_2 @ w_2.T$ times σ' . Outer products summed over the samples: $a_1.T @ \text{delta}_2$, $x.T @ \text{delta}_1$; the bias gradients are column sums.

What we add today

A list of layers instead of two named ones; any hidden activation with its derivative; three output heads (linear, sigmoid, softmax) with their cost functions; ℓ_2 and ℓ_1 penalties; minibatches; a gradient check. The two delta lines are Eqs. (8.69) and (8.66) and will not change.

Pause and think 1: where is the sum over k in the template?

Question

The fan-out rule says that the error of hidden node j is $\delta_j^1 = (\sum_k \delta_k^2 w_{jk}^2) \sigma'(z_j^1)$, a sum over the nodes of the next layer. The template has no loop and no sum – only `delta_1 = (delta_2 @ w_2.T) * a_1 * (1 - a_1)`. Where is the sum? What are the shapes if there are n samples, M_1 hidden nodes and M_2 output nodes – and over what does `x.T @ delta_1` sum?

Pause and think 1: where is the sum over k in the template?

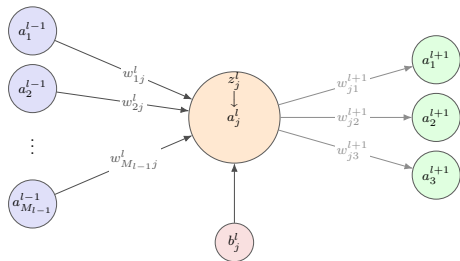
Question

The fan-out rule says that the error of hidden node j is $\delta_j^1 = (\sum_k \delta_k^2 w_{jk}^2) \sigma'(z_j^1)$, a sum over the nodes of the next layer. The template has no loop and no sum – only `delta_1 = (delta_2 @ w_2.T) * a_1 * (1 - a_1)`. Where is the sum? What are the shapes if there are n samples, M_1 hidden nodes and M_2 output nodes – and over what does `x.T @ delta_1` sum?

Answer

In the matrix product. `delta_2` has shape (n, M_2) and `w_2` shape (M_1, M_2) , so `delta_2 @ w_2.T` has shape (n, M_1) with entry $(i, j) = \sum_k \delta_{ik}^2 w_{jk}^2$: the sum over the receiving nodes, for every sample at once. The template was written for $M_2 = 1$, where the sum has one term, but the line is already Eq. (8.66) in full – which is why it survives unchanged. `x.T @ delta_1` has the shape of W^1 , (features, M_1), and sums over the *samples*: entry $(i, j) = \sum_s x_{si} \delta_{sj}^1$, the outer products of Eq. (8.55) added up. Two sums, two matrix products: one over the next layer, one over the data.

Dense layers: the notation and the two elementary derivatives (Section 8.5, Eqs. 8.18–8.21, 8.57)



$$z_j^l = \sum_{i=1}^{M_{l-1}} w_{ij}^l a_i^{l-1} + b_j^l, \quad a_j^l = f(z_j^l)$$

Book Figure 8.6: node j of layer l . The superscript of w_{ij}^l names the layer it *arrives at*, the first index the node it *leaves*.

Layers $l = 0$ (inputs, $a_i^0 = x_i$) to $l = L$ (outputs, $\tilde{y} = a^L$), M_l nodes in layer l . Eq. (8.18): $z_j^l = \sum_{i=1}^{M_{l-1}} w_{ij}^l a_i^{l-1} + b_j^l$, $a_j^l = f(z_j^l)$; vector form (8.20) $z^l = (W^l)^T a^{l-1} + b^l$; and for n samples in the rows of A^{l-1} , Eq. (8.21),

$$Z^l = A^{l-1} W^l + b^l, \quad A^l = f(Z^l),$$

with W^l of shape (M_{l-1}, M_l) *untransposed* – the template's convention; one multiplication per weight.

Everything follows from two derivatives of (8.18), Eq. (8.57)

$\partial z_j^l / \partial w_{ij}^l = a_i^{l-1}$ and $\partial z_j^l / \partial a_i^{l-1} = w_{ij}^l$, together with $\partial a_j^l / \partial z_j^l = f'(z_j^l)$ – for the sigmoid $a_j^l(1 - a_j^l)$, Eq. (8.22). The chain rule (1.51) does the rest.

The output layer: define the error and read it (Eqs. 8.58–8.60)

Squared-error cost (8.32), $C = \frac{1}{2} \sum_j (a_j^L - y_j)^2$ for one sample. A weight of the last layer affects C only through z_j^L :

$$\frac{\partial C}{\partial w_{ij}^L} = (a_j^L - y_j) \frac{\partial a_j^L}{\partial z_j^L} \frac{\partial z_j^L}{\partial w_{ij}^L} = (a_j^L - y_j) f'(z_j^L) a_i^{L-1} \quad (8.58).$$

Name the part that does not depend on i : the error of node j , Eq. (8.59)

$$\delta_j^L = f'(z_j^L) \frac{\partial C}{\partial a_j^L}, \quad \delta^L = f'(z^L) \circ \frac{\partial C}{\partial a^L},$$

◦ the Hadamard (element-wise) product, and (8.58) becomes $\partial C / \partial w_{ij}^L = \delta_j^L a_i^{L-1}$, Eq. (8.60): the scalar rule with two indices.

Read it, do not just record it

The second factor: how fast the cost changes with output j – if the cost hardly depends on that output, δ_j^L is small, as it should be. The first factor: how fast the activation changes at the value z_j^L *actually attained* – a saturated unit, $f' \approx 0$, contributes almost nothing *however wrong it is*. That second observation is the seed of the vanishing-gradient problem (Section 8.9) and the reason Part 3 pairs each output activation with a particular loss.

The error is the bias gradient, and it propagates backwards (Eqs. 8.61–8.64)

Two faces of δ . Since C depends on z_j^L only through a_j^L ,

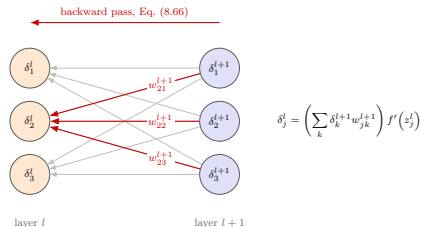
$$\delta_j^L = \frac{\partial C}{\partial z_j^L} = \frac{\partial C}{\partial a_j^L} \frac{\partial a_j^L}{\partial z_j^L} \quad (8.61),$$

and $\partial z_j^L / \partial b_j^L = 1$ gives $\delta_j^L = \partial C / \partial b_j^L$, Eq. (8.62): *the error of a node is the derivative of the cost with respect to its bias* – because the bias enters z additively. Hence one symbol for both.

A general layer. Define $\delta_j^l = \partial C / \partial z_j^l$. C depends on z_j^l only through the pre-activations z_k^{l+1} of the next layer, so the chain rule gives Eq. (8.63),

$$\delta_j^l = \sum_k \frac{\partial C}{\partial z_k^{l+1}} \frac{\partial z_k^{l+1}}{\partial z_j^l} = \sum_k \delta_k^{l+1} \frac{\partial z_k^{l+1}}{\partial z_j^l},$$

and from $z_k^{l+1} = \sum_i w_{ik}^{l+1} a_i^l + b_k^{l+1}$, $a_i^l = f(z_i^l)$:
 $\partial z_k^{l+1} / \partial z_j^l = w_{jk}^{l+1} f'(z_j^l)$.



Book Figure 8.13: Figure 8.6 with the arrows reversed.

The backpropagation equation, Eq. (8.64)

$$\delta_j^l = \left(\sum_k \delta_k^{l+1} w_{jk}^{l+1} \right) f'(z_j^l), \quad \delta^l = (W^{l+1} \delta^{l+1}) \circ f'(z^l).$$

The error at a layer is the error above, pulled back through the *transpose* of the weights (the sum runs over the *second* index) and modulated by the local derivative. Same graph, arrows reversed.

The four equations of backpropagation (Section 8.8.4, Eqs. 8.65–8.69)

Everything rests on these

$$\delta^L = f'(z^L) \circ \partial C / \partial a^L \quad (8.65) \text{ start}$$

$$\delta^l = (W^{l+1} \delta^{l+1}) \circ f'(z^l) \quad (8.66) \text{ carry back}$$

$$\partial C / \partial b_j^l = \delta_j^l \quad (8.67) \text{ bias}$$

$$\partial C / \partial w_{ij}^l = \delta_j^l a_i^{l-1} \quad (8.68) \text{ weight}$$

Every quantity on the right is already available: the z^l and a^l were computed in the forward pass and need only be stored, and f' is a cheap element-wise function. Equation (8.68) is the outer product $a^{l-1}(\delta^l)^T$, the shape of W^l .

A simplification worth knowing, Eq. (8.69)

For the natural pairings of output activation and loss the first equation collapses. Linear output and half-squared error: $f' = 1$, $\partial C / \partial a_j^L = a_j^L - y_j$, so $\delta^L = a^L - y$. Softmax output and multiclass cross entropy: the derivative of the softmax and the derivative of the logarithm *cancel* and the same equation holds – Part 3 shows it, and the sigmoid + cross entropy of Chapter 5 is the case $K = 2$. In all three the output error is prediction minus target, with no f' to shrink it: the deeper reason for pairing the softmax with the cross entropy, and why one line of code serves all heads.

Where the three small networks sit

Eq. (8.43) is (8.66) with $M_{l+1} = 1$; Eqs. (8.52)–(8.55) are (8.66)–(8.68) with $M_2 = 1$; the template's $\text{delta_2} = a_2 - y$ is (8.69). Nothing in Part 1 has to be unlearned.

Pause and think 2: the four equations for n samples at once

Question

Equations (8.65)–(8.68) are written for one sample, with δ^l a column of length M_l . The code keeps the n samples in rows: A^l and Z^l have shape (n, M_l) , Eq. (8.21). Write the four equations for the matrices Δ^l of shape (n, M_l) – which products, which transposes, and which sums over the samples? (Hint: the cost is a sum over the samples.)

Pause and think 2: the four equations for n samples at once

Question

Equations (8.65)–(8.68) are written for one sample, with δ^l a column of length M_l . The code keeps the n samples in rows: A^l and Z^l have shape (n, M_l) , Eq. (8.21). Write the four equations for the matrices Δ^l of shape (n, M_l) – which products, which transposes, and which sums over the samples? (Hint: the cost is a sum over the samples.)

Answer

$\Delta^L = f'(Z^L) \circ \partial C / \partial A^L$, the shape of the output ($A^L = Y$ for the natural pairings). Carrying it back, $W^{l+1} \delta^{l+1}$ for every row becomes $\Delta^l = (\Delta^{l+1} (W^{l+1})^T) \circ f'(Z^l)$, shape $(n, M_{l+1}) \times (M_{l+1}, M_l) = (n, M_l)$ – the template's `delta_2 @ w_2.T`. The cost is a sum (or mean) over the samples, so the gradients are summed: $\partial C / \partial W^l = (A^{l-1})^T \Delta^l$, shape (M_{l-1}, M_l) = the shape of W^l , each entry a sum of outer products; $\partial C / \partial b^l$ is the column sum of Δ^l . Two transposes in two different places: $(W^{l+1})^T$ pulls the error back through the next layer, $(A^{l-1})^T$ sums over the data. With the mean cost, divide Δ^L by n once and everything downstream inherits it.

What it costs, and what it is (Section 8.8.4)

Two forward passes, whatever the number of parameters

Equation (8.66) costs one matrix-vector product per layer, exactly as the forward pass does; (8.68) one outer product per layer. The whole gradient with respect to *all* parameters costs about twice a forward pass, independently of how many parameters there are – Proposition 4.2, the cheap gradient principle of reverse-mode automatic differentiation, and backpropagation *is* that algorithm specialised to a layered network. Finite differences, Eq. (4.102), need one forward pass per parameter: for 10^6 weights, a second against a fortnight.

Why reverse and not forward

Section 5.8.2: forward mode needs one sweep per parameter, reverse mode one sweep per output. A network has 10^4 – 10^9 parameters and one cost. Training asks the reverse-mode question, every time.

Always verify a hand-written backward pass (notebox, Section 8.8.5)

Perturb one weight by $h \approx 10^{-6}$, evaluate the cost twice, compare the central difference with the analytical gradient: the relative discrepancy should be about 10^{-6} or smaller. A wrong gradient does not announce itself – the network still trains, just to the wrong place, or slightly worse than it should. Our code (Part 4) passes at 10^{-8} to 10^{-6} for every head and both activations, and agrees with `jax.grad` to 4×10^{-16} .

What `loss.backward()` does

Exactly this: reverse-mode AD over the graph the forward pass recorded. PyTorch's five lines (Section 8.14) are (8.21), (8.65)–(8.68) and (8.70); knowing what they do is the point of deriving them.

The algorithm (Section 8.8.5)

Set-up

1. The architecture: inputs, outputs, hidden layers and their widths.
2. Activation functions for the hidden layers, and for the output layer – dictated by the *range* of the target.
3. The cost function – dictated by the *distribution* of the target (Section 3.6) – and any penalties with their λ .
4. The optimiser of Chapter 4 and its learning rate γ .
5. Initialise weights and biases (Section 8.10, next frame).

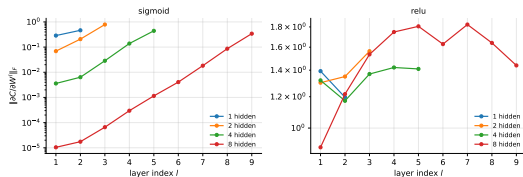
Steps 1–4 are `feed_forward` and `backpropagation` in Part 4, step 5 is one line, and the loop over epochs and minibatches is `train`. The set-up is where the choices are – and where Project 2 will ask you to argue.

Each iteration, on a minibatch of M samples

1. *Forward pass*: $A^0 = X$, then (8.21) for $l = 1, \dots, L$, storing every Z^l and A^l .
2. *Output error*: Δ^L from (8.65), or $A^L - Y$ for the natural pairings (8.69).
3. *Backward pass*: Δ^l for $l = L - 1, \dots, 1$ from (8.66).
4. *Gradients*: $(A^{l+1})^T \Delta^l$ and the column sums of Δ^l , (8.67)–(8.68).
5. *Update*: with plain gradient descent, Eq. (8.70), $w_{ij}^l \leftarrow w_{ij}^l - \gamma \delta_j^l a_i^{l-1}$, $b_j^l \leftarrow b_j^l - \gamma \delta_j^l$ – or momentum, RMSProp, Adam from week 38.

One pass over all n/M minibatches is an *epoch* (Section 4.7).

Two things the code must get right: the product of the recursion (8.9–8.10)



Book Figure 8.14: norm of the weight gradient per layer after one backward pass, 1–8 hidden layers of 30 units, digits data; sigmoid left, ReLU right.

Unrolling (8.66) from the output to layer l , Eq. (8.71):

$$\delta^l = \left[\prod_{k=l}^{L-1} W^{k+1} \text{diag}(f'(z^k)) \right] \delta^L,$$

a product of $L - l$ factors. Many factors below one shrink geometrically (*vanishing* gradients: the early layers never learn), above one grow geometrically (*exploding*). The sigmoid has $\sigma' \leq \frac{1}{4}$: ten layers give at best $4^{-10} \approx 10^{-6}$, last week's Pause and think 3. ReLU has $f' = 1$ on the active path – most of why deep networks became trainable.

Initialisation, Eqs. (8.72)–(8.74)

$\text{var}(z) = M_{l-1} \text{var}(w) \text{var}(a)$: to keep the signal of constant size forwards and backwards, draw w with variance $2/(M_{l-1} + M_l)$ (Xavier/Glorot) or $2/M_{l-1}$ for ReLU (He). All zeros: every unit computes the same thing forever (the symmetry of Section 8.7 must be broken). Too large: saturation, $f' \approx 0$, the $\gamma = 1$ run of week 40. Our `create_layers` uses $1/M_{l-1}$ (sigmoid) and $2/M_{l-1}$ (ReLU); with $\mathcal{N}(0, 1)$ weights and 784 inputs the MNIST network would start saturated.

Three tasks, three output layers, three cost functions – one output error (8.7, 8.11)

The cost (8.31): $C(\Theta) = \frac{1}{n} \sum_i L(y_i, \tilde{y}_i(\Theta)) + \lambda \Omega(\Theta)$. Section 3.6 decides the loss from the distribution of the target; the output activation is decided by its range.

Task	Output activation	Loss L (one sample)	$\partial L / \partial a^L$	δ^L
regression, $y \in \mathbb{R}$	linear, $a^L = z^L$	$\frac{1}{2}(a^L - y)^2$, Eq. (8.32)	$a^L - y$	$a^L - y$
binary, $y \in \{0, 1\}$	sigmoid, $a^L = \sigma(z^L)$	$-y \ln a^L - (1 - y) \ln(1 - a^L)$, (5.13)	$\frac{a^L - y}{a^L(1 - a^L)}$	$a^L - y$
K classes, y one-hot	softmax, $a_k^L = e^{z_k^L} / \sum_l e^{z_l^L}$ (8.75)	$-\sum_k y_k \ln a_k^L$, (5.28)	$-y_k / a_k^L$	$a^L - y$

The last column is Eq. (8.69) three times

Each loss is the negative log-likelihood of the distribution that fits the target (Gaussian, Bernoulli, categorical), each output activation is the mean of that distribution as a function of z , and for exactly these pairings the chain rule leaves *prediction minus target*. The next frames show the three cases.

What changes in the code

The output activation (three functions), the cost (three formulas) – and the backward pass starts with the *same* line `delta = (a[-1] - Y)/n` for all three. Everything behind it, (8.66)–(8.68), does not know which head is attached.

Regression: the squared error with ℓ_2 and ℓ_1 penalties (8.7, 8.12)

$$C(\Theta) = \frac{1}{2n} \sum_{i=1}^n (a_i^L - y_i)^2 + \lambda_2 \sum_l \sum_{ij} (w_{ij}^l)^2 + \lambda_1 \sum_l \sum_{ij} |w_{ij}^l|,$$

the Ridge (3.43) and Lasso (3.57) penalties of Chapter 3 applied to the weights of *every* layer; the first is called *weight decay* in the network literature. Biases are not penalised (Section 3.13 on the intercept applies: shifting the output should be free).

What they add to the backward pass

The data term gives $\delta^L = (a^L - y)/n$ and (8.66)–(8.68) as before. The penalties depend on the weights *directly*, not through the network, so they add to the weight gradient only:

$$\frac{\partial C}{\partial w_{ij}^l} = \delta_j^l a_i^{l-1} + 2\lambda_2 w_{ij}^l + \lambda_1 \text{sign}(w_{ij}^l),$$

one line in the code, the bias gradient unchanged. `sign` is the subgradient of $|w|$ from week 39's Lasso (zero at $w = 0$).

What they do

ℓ_2 : every weight is pulled towards zero in proportion to its size – the update $w \leftarrow (1 - 2\gamma\lambda_2)w - \gamma\delta a$ *decays* the weight before the gradient step, hence the name. ℓ_1 : a constant pull $\gamma\lambda_1$ whatever the size, so small weights reach exactly zero and stay there – sparse weights, as in Section 3.9. Both control the variance term of Section 2.11 for a model with more parameters than data; early stopping (8.12) does the same for free. Part 4 measures both on the Runge function.

Binary classification: the cross entropy and the cancellation (Section 5.3, 8.11)

Sigmoid output $a^L = \sigma(z^L) = p(y = 1 | x)$, targets $y \in \{0, 1\}$, the cross entropy (5.13) of one sample, $L = -y \ln a^L - (1 - y) \ln(1 - a^L)$: the negative log-likelihood of a Bernoulli variable.

The output error, Eq. (8.65) worked out

$$\frac{\partial L}{\partial a^L} = -\frac{y}{a^L} + \frac{1-y}{1-a^L} = \frac{a^L - y}{a^L(1-a^L)},$$

and $\sigma'(z^L) = a^L(1-a^L)$, so $\delta^L = \sigma'(z^L) \partial L / \partial a^L = a^L - y$. The σ' that shrinks every squared-error gradient is cancelled by the denominator of the cross entropy – Eq. (5.36) of week 39, now at the output of a network. Summed over the samples, the gradient of the last layer is $(A^{L-1})^T (a^L - y)$: Eq. (5.17) with the learned features A^{L-1} in place of X .

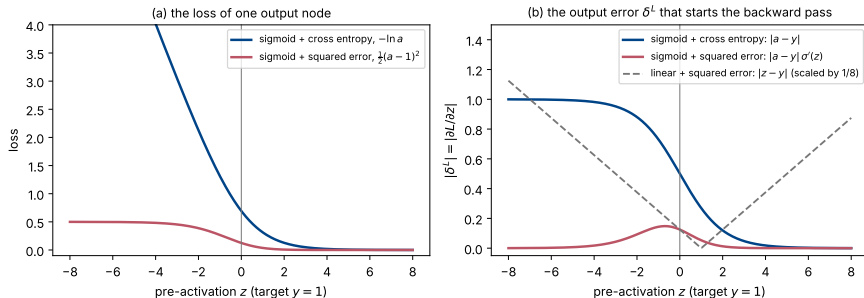
Logistic regression is the network with $L = 1$ (Section 8.11)

No hidden layer, one sigmoid output and the cross entropy: *exactly* Chapter 5, and our code reproduces it with `sizes = [p, 1]`. A classification network is logistic regression applied to the learned features a^{L-1} rather than to the raw inputs – Chapter 5 with a representation learner bolted on in front. That is the most useful way to think about what the hidden layers are for.

Two classes, two ways

One sigmoid node (probability of class 1) or two softmax nodes (both probabilities): identical models, since the softmax with $K = 2$ is the sigmoid of the difference of the two scores (Section 5.6).

Why not the squared error for classification? (week41_delta_heads.pdf)



One sigmoid output node, target $y = 1$, against the pre-activation z . **(a)** The cross entropy $-\ln \sigma(z)$ grows *linearly* for $z \rightarrow -\infty$; the squared error $\frac{1}{2}(\sigma(z) - 1)^2$ saturates at $\frac{1}{2}$. **(b)** The output error that starts the backward pass. Cross entropy: $|a - y| \rightarrow 1$, the more wrong the stronger the signal. Squared error with a sigmoid: $|a - y| \sigma'(z)$, which *vanishes* where the network is confidently wrong – at $z = -5$, $a = 0.0067$, the cross entropy gives $\delta^L = -0.993$ and the squared error -0.0066 , a factor 150 smaller. A wrong, saturated output node learns almost nothing from its mistake: the mechanism of Eq. (8.59), one layer deep. Linear output with the squared error (grey): $|z - y|$, no saturation, the right pairing for a real-valued target.

Pause and think 3: the softmax is shift-invariant: what does that do to the errors?

Question

The softmax (8.75) does not change if the same constant is added to every score z_k^L – Section 5.6 called it over-parametrised, and the stable implementation (5.30) uses it to subtract $\max_k z_k^L$. With the multiclass cross entropy and one-hot targets, what is $\sum_k \delta_k^L$ for one sample? What does that say about the bias gradients of the output layer, and about the direction in parameter space that the network can never learn?

Pause and think 3: the softmax is shift-invariant: what does that do to the errors?

Question

The softmax (8.75) does not change if the same constant is added to every score z_k^L – Section 5.6 called it over-parametrised, and the stable implementation (5.30) uses it to subtract $\max_k z_k^L$. With the multiclass cross entropy and one-hot targets, what is $\sum_k \delta_k^L$ for one sample? What does that say about the bias gradients of the output layer, and about the direction in parameter space that the network can never learn?

Answer

$\sum_k \delta_k^L = \sum_k (a_k^L - y_k) = 1 - 1 = 0$: the probabilities sum to one and so does the one-hot target, so the K output errors of every sample sum to zero. By (8.67) the K output-bias gradients sum to zero as well, and so does every row $\sum_k \partial C / \partial w_{jk}^L = a_j^{L-1} \sum_k \delta_k^L$: adding the same constant to all K biases, or the same vector to the K columns of W^L , changes neither the cost nor its gradient. The gradient has no component along that direction, gradient descent leaves it where the initialisation put it, and an ℓ_2 penalty selects the smallest representative (Section 5.6). One of the K output nodes is redundant – for $K = 2$ one sigmoid node suffices – and nothing goes wrong by keeping it.

K classes: the softmax and the multiclass cross entropy (Sections 5.6, 8.11)

The output layer, Eq. (8.75)

K output nodes, scores z_k^L , and

$$a_k^L = \frac{\exp(z_k^L)}{\sum_{l=0}^{K-1} \exp(z_l^L)}, \quad a_k^L > 0, \quad \sum_k a_k^L = 1,$$

read as $p(C = k \mid \mathbf{x})$. Never as written: subtract $\max_k z_k^L$ first, Eq. (5.30) – the exponents are then ≤ 0 , nothing overflows, and the probabilities are unchanged.

The targets and the loss, Eq. (5.28)

Label 5 of ten becomes the one-hot vector $\mathbf{y} = (0, 0, 0, 0, 0, 1, 0, 0, 0, 0)$; the loss of one sample is $L = -\sum_{k=0}^{K-1} y_k \ln a_k^L = -\ln a_{k^*}^L$: only the probability assigned to the *correct* class k^* is punished. Negative log-likelihood of a categorical variable; Eq. (5.13) when $K = 2$.

The derivative of the softmax

Every output depends on every score, so (8.65) needs the full Jacobian. With $S = \sum_l e^{z_l}$,

$$\frac{\partial a_k}{\partial z_j} = \frac{e^{z_k} \delta_{kj}}{S} - \frac{e^{z_k} e^{z_j}}{S^2} = a_k (\delta_{kj} - a_j),$$

δ_{kj} the Kronecker delta. Not diagonal: the Hadamard form of (8.65) assumed that a_j depends on z_j only.

The cancellation, Eq. (8.69) again

$$\begin{aligned} \delta_j^L &= \sum_k \frac{\partial L}{\partial a_k} \frac{\partial a_k}{\partial z_j} = \sum_k \left(-\frac{y_k}{a_k}\right) a_k (\delta_{kj} - a_j) \\ &= -y_j + a_j \sum_k y_k = a_j^L - y_j. \end{aligned}$$

The a_k of the Jacobian cancels the $1/a_k$ of the logarithm and $\sum_k y_k = 1$ does the rest: prediction minus target, for the third time.

From the template to a general network, I: layers and the forward pass (week41.ipynb)

```
ACTIVATIONS = {"sigmoid": (sigmoid, sigmoid_prime), "relu": (relu, relu_prime),
               "identity": (identity, identity_prime)}
OUTPUT = {"mse": identity, "binary": sigmoid, "softmax": softmax} # the three heads

def create_layers(sizes, hidden_activation="sigmoid", rng=None):
    """[784, 100, 10] -> two (W, b) pairs, W of shape (inputs, outputs)."""
    rng = np.random.default_rng(2026) if rng is None else rng
    layers = []
    for n_in, n_out in zip(sizes[:-1], sizes[1:]):
        scale = np.sqrt(2.0/n_in) if hidden_activation == "relu" else np.sqrt(1.0/n_in)
        layers.append((rng.normal(0.0, scale, (n_in, n_out)), np.zeros(n_out) + 0.01))
    return layers

def feed_forward(X, layers, hidden_activation="sigmoid", head="mse"):
    """Eq. (8.21) layer by layer; returns the lists z and a (a[0] = X)."""
    f, _ = ACTIVATIONS[hidden_activation]
    a, z = [X], []
    for l, (W, b) in enumerate(layers):
        z1 = a[-1] @ W + b # the template's x @ w_1 + b_1
        z.append(z1)
        a.append(OUTPUT[head](z1) if l == len(layers) - 1 else f(z1))
    return z, a
```

What changed, what did not

The two named layers `w_1`, `w_2` became a *list* of (W, b) pairs; the forward pass is the template's two lines inside a loop, and *keeps every z^l and a^l* because (8.66) needs $f'(z^l)$ and (8.68) needs a^{l-1} . The last layer applies the output head instead of the hidden activation. Samples in rows, W of shape (inputs, outputs), exactly as before.

Initialisation and the activations

Variance $1/M_{l-1}$ for the sigmoid, $2/M_{l-1}$ (He, 8.74) for ReLU; biases 0.01 as in the template. The template's `randn` has variance 1: fine for one input, saturating for 784. sigmoid is evaluated through $e^{-|z|}$ so that it never overflows; softmax subtracts the row maximum, Eq. (5.30).

From the template to a general network, II: the backward pass and the training loop

```
def backpropagation(X, Y, layers, hidden_activation="sigmoid", head="mse", lmbd2=0.0,
                    lmbd1=0.0):
    """Eqs. (8.65)-(8.68) for all samples at once; returns a list of (dW, db)."""
    _, fp = ACTIVATIONS[hidden_activation]
    n = X.shape[0]
    z, a = feed_forward(X, layers, hidden_activation, head)
    delta = (a[-1] - Y)/n # Eq. (8.69): the same line for all three heads
    grads = [None]*len(layers)
    for l in range(len(layers) - 1, -1, -1):
        W, _ = layers[l]
        dW = a[l].T @ delta # Eq. (8.68): outer products, summed over the samples
        dW += 2.0*lmbd2*W + lmbd1*np.sign(W) # the penalties of Section 8.12 (weights only)
        db = np.sum(delta, axis=0) # Eq. (8.67)
        grads[l] = (dW, db)
        if l > 0:
            delta = (delta @ W.T) * fp(z[l - 1]) # Eq. (8.66): pull back through W,
            # modulate by f'
    return grads

def train(X, Y, layers, hidden_activation, head, gamma=0.1, epochs=100, batch_size=None,
          lmbd2=0.0, lmbd1=0.0, rng=None, ...):
    for epoch in range(epochs):
        order = rng.permutation(n) # reshuffle, then minibatches of M
        for s in range(0, n, M):
            idx = order[s:s + M]
            grads = backpropagation(X[idx], Y[idx], layers, hidden_activation, head, lmbd2,
                                    lmbd1)
            layers = [(W - gamma*dW, b - gamma*db) for (W, b), (dW, db) in zip(layers,
                                        grads)] # (8.70)
```

Line by line

$\text{delta} = (a[-1] - Y)/n$: Eq. (8.69) for the mean cost; every gradient downstream carries the $1/n$. $a[l].T @ \text{delta}$: the outer products of (8.68) summed over the samples, the shape of W . $\text{delta} @ W.T$: the sum over the receiving nodes of (8.66), pulled back through the transpose, times f' of the *stored* pre-activation ($z[0]$ belongs to the first layer, hence the shift). $l > 0$ stops at the inputs, whose adjoints we never need.

The template, recovered

Two layers, $x = 4$, $y = 9$, seed 0, $\gamma = 0.1$: cost 36.895631, 0.426990, 0.004939, 0.000000 after 0/10/20/49 iterations, $w^1 = 1.7958$, $w^2 = 4.695$ – every digit of week 40. For momentum or Adam replace the one-line update by week 38's `optimiser_step`.

Checking the gradient: finite differences and `jax.grad`, three heads, two penalties

```
def jax_cost(params, X, Y, hidden_activation, head, lmbd2, lmbd1):
    """The same cost in jax.numpy: the forward pass and nothing else."""
    f = {"sigmoid": jax.nn.sigmoid, "relu": jax.nn.relu}[hidden_activation]
    a = X
    for l, (W, b) in enumerate(params):
        z = a @ W + b
        a = f(z) if l < len(params) - 1 else OUTPUT_JAX[head](z)
    data = {"mse": 0.5*jnp.sum((a - Y)**2)/n, "binary": ...,
           "softmax": -jnp.mean(jnp.sum(Y*jnp.log(a), axis=1))}[head]
    return data + sum(lmbd2*jnp.sum(W**2) + lmbd1*jnp.sum(jnp.abs(W))) for W, _
        in params)

g_jax = jax.grad(jax_cost)(params, X, Y, act, head, 1e-3, 1e-3) # a list of (dW
, db)
```

```
finite differences jax.grad
mse sigmoid [5, 7, 6, 2] 8.48e-08 1.11e-16
mse relu [5, 7, 6, 2] 5.80e-08 4.44e-16
binary sigmoid [5, 7, 6, 1] 7.76e-08 2.78e-17
binary relu [5, 7, 6, 1] 1.19e-08 5.55e-17
softmax sigmoid [5, 7, 6, 3] 6.32e-07 5.55e-17
softmax relu [5, 7, 6, 3] 8.04e-08 2.78e-17
```

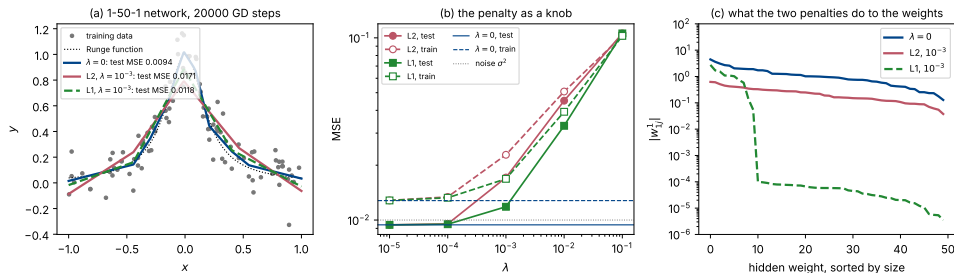
Two independent referees

Finite differences, Eq. (4.102): twenty random weights per layer, $h = 10^{-6}$, worst relative discrepancy 10^{-8} – 10^{-6} – the accuracy of the difference scheme itself (week 37: truncation against round-off). `jax.grad`: the forward pass written once more in `jax.numpy`, differentiated by reverse-mode AD; the largest absolute difference over all (W, b) is 4×10^{-16} , machine precision. Two hidden layers of 7 and 6, both activations, both penalties on, all three heads: Eqs. (8.65)–(8.68) are transcribed correctly.

Which one to keep

The JAX version is *shorter*: it contains no backward pass at all. Project 2 asks for both – the hand-written one so that you know what `jax.grad` does, the AD one because it is what you will use afterwards. Keep the finite-difference check; it costs nothing and catches the index errors that AD does not need to make.

Example 1, regression: the Runge function of Project 1 with a 1–50–1 network



$n = 100$, $\sigma = 0.1$, seed 2026, a separate test set of 100 points; one hidden layer of 50 ReLU units, linear output, half-squared error, plain gradient descent, $\gamma = 0.1$, 20 000 steps. (a) Without penalty: train MSE 0.0128, test 0.0094 – the noise variance is 0.01, the network has found the function. With $\lambda = 10^{-3}$: ℓ_2 0.0228/0.0171, ℓ_1 0.0168/0.0118. (b) Both penalties only cost here (nothing changes up to 10^{-4} ; at 10^{-1} the weights are driven to zero and the network predicts the mean, MSE 0.1025 = $\text{var}(y)$): with $n = 100$ and 20 000 steps the unpenalised network is not overfitting. (c) ℓ_1 sends 40 of the 50 hidden weights below 10^{-3} and keeps ten; ℓ_2 shrinks all fifty by a factor and zeroes none. With sigmoid units (notebook): $\gamma = 0.1$ reaches only 0.0237/0.0184 in 20 000 steps, $\gamma = 0.2$ gives 0.0169/0.0124, and $\gamma = 0.5$ collapses to 0.1024/0.1057: the hidden layer saturates in the first steps and predicts the mean – the $\gamma = 1$ run of week 40 with fifty neurons. ReLU at $\gamma = 0.5$ diverges too (0.0815); at 0.1 it is at the noise floor after a thousand steps.

Pause and think 4: forty weights at 10^{-4} : what happened to those hidden nodes?

Question

With the ℓ_1 penalty at $\lambda = 10^{-3}$, forty of the fifty hidden weights w_{ij}^1 end up below 10^{-3} and the fit is nearly as good as without penalty. What does a hidden ReLU node with $w_{ij}^1 = 0$ compute, where does its contribution go, and what has the penalty effectively done to the architecture? Why does ℓ_2 not do the same?

Pause and think 4: forty weights at 10^{-4} : what happened to those hidden nodes?

Question

With the ℓ_1 penalty at $\lambda = 10^{-3}$, forty of the fifty hidden weights w_{ij}^1 end up below 10^{-3} and the fit is nearly as good as without penalty. What does a hidden ReLU node with $w_{ij}^1 = 0$ compute, where does its contribution go, and what has the penalty effectively done to the architecture? Why does ℓ_2 not do the same?

Answer

A node with $w_{ij}^1 = 0$ computes $\max(0, b_j^1)$, a *constant*, and the output layer adds $w_j^2 \max(0, b_j^1)$ to its bias: forty constants fold into one number, b^2 , so the trained network is a 1–10–1 network. Ten surviving nodes supply ten kinks, and a piecewise-linear function with ten kinks follows the Runge function to the noise level (the constructive theorem of Section 8.4.1 in one dimension). The ℓ_1 penalty has *selected the architecture*, as the Lasso selected features in Chapter 3: the subgradient pull $\gamma\lambda_1$ is the same for a weight of 10^{-3} as for one of 1, so small weights hit zero and stay. The ℓ_2 pull $2\gamma\lambda_2 w$ vanishes with w : every weight shrinks by the same *factor* and none reaches zero – panel (c), a parallel line below the unpenalised one. Which would you rather report in Project 2: fifty units of which ten matter, or ten units?

Example 2, binary classification: 569 tumours, 30 features, no hidden layer against one

The Wisconsin breast-cancer data (`sklearn`): 569 samples, 30 standardised features, target malignant/benign. Sigmoid output, cross entropy, ℓ_2 with $\lambda = 10^{-4}$, minibatches of 32, $\gamma = 0.1$, 300 epochs, seed 2026, 455 training and 114 test samples.

Network	parameters	train accuracy	test accuracy	final cost
30-1: logistic regression, Chapter 5	31	0.9868 (6 wrong)	0.9825 (2 wrong)	0.0522
30-16-1: one hidden sigmoid layer	513	0.9912 (4 wrong)	0.9737 (3 wrong)	0.0500

The same code, `sizes = [30, 1]`

Logistic regression is the network with $L = 1$: one call of `create_layers` and the backward pass of the last frame with the loop running once. Week 39's gradient $X^T(p - y)$ is `a[0].T @ delta` with n divided out.

A hidden layer is not automatically better

Sixteen times the parameters, a lower training cost, one more test error out of 114. The classes are nearly linearly separable in the standardised features, so there is no representation to learn, and the difference of one sample is well inside the variance of a 114-sample test set (Section 2.10: a binomial standard deviation of ≈ 1.5 samples). Measure with cross-validation before you believe either number – Project 2 will ask for exactly that comparison.

Example 3: MNIST, the full set – 70 000 handwritten digits (Section 8.14)

The data

60 000 training and 10 000 test images of 28×28 grey-scale pixels, labels 0–9 (LeCun, Cortes, Burges, 1998; the course has it as `mnist.npz`, otherwise `fetch_openml("mnist_784")`). Each image is flattened to a row of 784 features in $[0, 1]$: the spatial structure is thrown away – convolutional networks (Chapter 10) keep it. The labels become one-hot rows, Y of shape $(60\,000, 10)$.

The network

`sizes = [784, 100, 10]`:
 $784 \cdot 100 + 100 + 100 \cdot 10 + 10 = 79\,510$ parameters;
sigmoid or ReLU hidden layer, softmax output,
multiclass cross entropy. Plain gradient descent on
minibatches of $M = 100$: 600 updates per epoch,
 $\gamma = 0.5$ (sigmoid) and 0.1 (ReLU), 20 epochs.

What one epoch costs

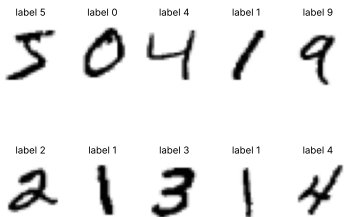
Forward pass: $2 \cdot 60\,000 \cdot (784 \cdot 100 + 100 \cdot 10) \approx 9.5 \cdot 10^9$ flops; the backward pass about twice that (one product to pull the error back, one outer product per layer, Section 8.8.4). Some 30 Gflop per epoch – one to two seconds of numpy on a laptop, 20 epochs in 20 s. Full-batch gradient descent would take the same time per *update* and make 20 of them instead of 12 000: week 38's argument for SGD, Eq. (4.38), at $n = 60\,000$.

What to expect

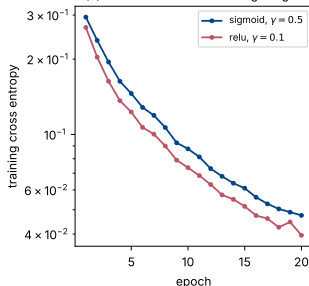
Every class a tenth of the data, so guessing gives 10%. Multinomial logistic regression on the raw pixels ($[784, 10]$) reaches about 92%; a one-hidden-layer network with 800 units and the cross entropy 98.4% (Simard et al. 2003); convolutional networks 99.7% and better.

MNIST: twenty epochs (week41_mnist.pdf)

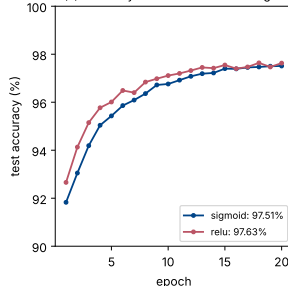
(a) ten of the 60 000 training images



(b) cost on the 60 000 training images

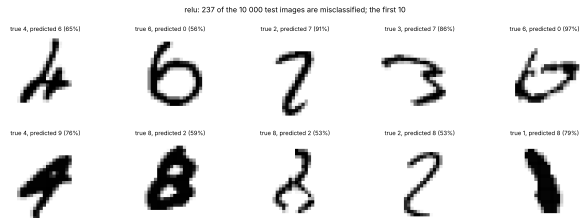


(c) accuracy on the 10 000 test images



(b) The training cross entropy falls from 0.29 (sigmoid) and 0.27 (ReLU) after the first epoch to 0.048 and 0.040 after twenty, still falling. **(c)** Test accuracy 91.8%/92.7% after one epoch – already the logistic-regression level – 96.8%/97.1% after ten, 97.51% (sigmoid) and 97.63% (ReLU) after twenty, on images the network has never seen. Training accuracy 98.74% and 98.97%: the gap is the variance term, small because 60 000 samples stand against 79 510 parameters. ReLU is ahead by a few tenths of a percent at every epoch with a five times smaller γ ; with one hidden layer the vanishing-gradient argument has little room, Figure 8.14 shows what happens with eight. Twenty-one and seventeen seconds of numpy, seed 2026 – the notebook reproduces every number.

MNIST: the 237 the ReLU network gets wrong (week41_mnist_errors.pdf)



The confusion matrix, off-diagonal highlights

Rows true, columns predicted, of 10 000: $4 \rightarrow 9$ 19 times, $7 \rightarrow 2$ 10, $3 \rightarrow 2$ 8, $7 \rightarrow 9$ 8, $9 \rightarrow 3$ 6, $5 \rightarrow 3$ 6. The diagonal holds 9763. Fours with a closed top, sevens with a bar, threes and eights with a thin waist – the errors are the pairs a person hesitates on, and the softmax probabilities of the wrong answers (53%–97%) show the network hesitating too.

What the probabilities are for

A softmax output is a *distribution* over the classes, not a label. The 53% “2” with 8 in second place is a different mistake from the 97% “0”: the first is an honest doubt, the second a wrong feature. Project 2 will ask you to look at your errors this way, not only to count them.

Pause and think 5: is 97.6% good – and what would you change first?

Question

Our 784–100–10 network makes 237 errors on the 10 000 test images after twenty epochs of plain minibatch gradient descent; the training cost is still falling and the training accuracy is 98.97%. Which of the following would you try first, and what do you expect from each: more epochs; a validation set; momentum or Adam; a second hidden layer; more hidden units; ℓ_2 ; a convolutional network?

Pause and think 5: is 97.6% good – and what would you change first?

Question

Our 784–100–10 network makes 237 errors on the 10 000 test images after twenty epochs of plain minibatch gradient descent; the training cost is still falling and the training accuracy is 98.97%. Which of the following would you try first, and what do you expect from each: more epochs; a validation set; momentum or Adam; a second hidden layer; more hidden units; ℓ_2 ; a convolutional network?

Answer

More epochs first, since nothing has converged – but with a *validation set* split off the training data to decide when to stop, because the test set must not choose hyperparameters (Section 3.14); the gap 98.97% against 97.63% is small but will open. *Adam or momentum* buy the same accuracy in fewer epochs, not a higher ceiling. *More units or a second layer* raise the ceiling of this family (800 units reach 98.4%) at the price of more variance, which ℓ_2 or early stopping then control; a second ReLU layer with He initialisation is cheap, with sigmoids it is where Figure 8.14 starts to bite. The errors point elsewhere: $4 \rightarrow 9$ and $7 \rightarrow 2$ are *shape* confusions, and a network that sees each pixel as an independent feature cannot know that a stroke moved one pixel to the right is the same stroke. That knowledge is what convolutions add – Chapter 10, and the reason they reach 99.7%. Matching the architecture to the data beats tuning the optimiser.

Where this is going: Project 2, the frameworks, and tomorrow

Project 2 (to be published after the Project 1 deadline)

Exactly this network – your own `feed_forward`, `backpropagation` and `train`, with the gradient check – on the regression problem of Project 1 and on a classification problem, compared with the linear and logistic models you already have, with the optimisers of week 38 and the resampling of week 36. Everything on these slides is a building block; nothing on them is the project.

The frameworks, Section 8.14

```
sklearn.neural_network.MLPClassifier; in PyTorch
nn.Sequential(nn.Linear(784, 100), nn.ReLU(),
nn.Linear(100, 10)), nn.CrossEntropyLoss() (softmax
folded in), loss.backward() – Eqs. (8.65)–(8.68) – and
optimiser.step(), Eq. (8.70). Week 42 and 43.
```

Tomorrow: one case, the feed-forward part

The structure of this week's exercises: one layer by hand with the shapes, a list of layers, an activation function per layer, many samples at once with the transposed convention, and a prediction on a real data set before any training – in `week41tuesday.ipynb` on the digits data, so that you do it once with us and once on your own on the iris data of `exercisesweek41.ipynb`. Deadline **Friday October 9**.

Next Monday, week 42

The backward pass in the exercises (your own and `jax.grad`), training the network, and the practical side of Sections 8.9–8.12: activation functions and initialisation measured, dropout, batch normalisation, hyperparameter search – and the start of Project 2.

Summary of week 41

- ▶ **The reminder:** $\delta = \partial C / \partial z$, gradient = error times the signal that entered the weight, the recursion $\delta^1 = \delta^2 w^2 \sigma'(z^1)$; the only thing missing was the sum over the nodes a node feeds – and the template already had it, inside `delta_2 @ w_2.T`.
- ▶ **The general case:** two elementary derivatives (8.57) and the chain rule give the output error (8.59), the error as bias gradient (8.62) and the backpropagation equation (8.64); the four equations (8.65)–(8.68); two forward passes for the whole gradient (Proposition 4.2); the algorithm; the product of the recursion (8.71) and the initialisation (8.73–8.74) that keeps it tame.
- ▶ **Cost functions:** the loss follows the distribution of the target, the output activation its range; for linear + squared error, sigmoid + cross entropy and softmax + multiclass cross entropy the output error is $a^L - y$ (8.69) – the f' cancels, which the squared error with a sigmoid does not (a factor 150 at $z = -5$); ℓ_2 decays weights, ℓ_1 zeroes them, biases are exempt; the softmax errors sum to zero.
- ▶ **The code:** the template with a list of layers, three heads, two penalties, minibatches; finite differences 10^{-8} – 10^{-6} , `jax.grad` 4×10^{-16} ; Runge at the noise floor with ReLU (ℓ_1 keeps ten of fifty units), logistic regression as the network with $L = 1$, and MNIST at 97.5%–97.6% in twenty seconds – with the errors a person would make.

Tomorrow and the deadline

Tuesday: your questions, then *one* case – the feed-forward part, step by step, on the digits data. Exercises week 41 (the same steps on the iris data, with `jax.grad` as the optional last step) are due **Friday October 9 at midnight**.